On the Stability of Code Generation under Semantically Equivalent Intermediate Explanations

Mingyang Xie
MIB Laboratory, Queen's University
21mx4@queensu.ca

Xiangyu Jiao
MIB Laboratory, Queen's University
20xj6@queensu.ca

Chuyi Qin
MIB Laboratory, Queen's University
21ciq@queensu.ca

Ting Hu*
MIB Laboratory, Queen's University
ting.hu@queensu.ca

Abstract

Large language models (LLMs) achieve strong performance in code generation, yet the stability of their behavior in multi stage coding pipelines remains insufficiently understood. In particular, it is unclear whether semantically equivalent, model generated explanations, often treated as reliable intermediate representations, lead to consistent executable code. We present an evaluation framework that factorizes the pipeline into Problem to Explanation to Code to Execution, and quantify stability at both the explanation and execution levels. We curate a benchmark of algorithmic problems spanning multiple difficulty tiers and classical machine learning algorithms, selected to balance practical relevance and reduced memorization likelihood. For each problem, the model generates k semantically equivalent explanations for each temperature in isolated interactions, each used in a separate interaction to generate code. We evaluate functional correctness with an online judge style test harness, and introduce a structure aware, sequence sensitive semantic consistency metric designed for code reconstructability rather than surface overlap. Our results show that small variations in intermediate explanations can be amplified into large differences in code correctness, revealing a distinct and practically important instability mode beyond user prompt reformulations.

1 Introduction

Large language models (LLMs) have demonstrated strong performance in code generation and are increasingly used in practical development workflows. Most existing evaluations assess functional correctness under a fixed problem description, typically via unit tests, online-judge-style scoring, or metrics such as pass@k [Chen et al., 2021, Liu et al., 2023, Hendrycks et al., 2021]. While these benchmarks have been valuable for tracking progress, they largely treat code generation as a single-step mapping from a problem statement to executable code. As a result, they provide limited insight into the reliability of multi-stage LLM-based coding pipelines in which intermediate representations play an explicit role.

In practical LLM-assisted programming workflows, code generation is often embedded in pipelines that first produce and then consume intermediate artifacts, such as explanations, plans, or algorithmic sketches [Lei et al., 2025, Wei et al., 2022, Wang et al., 2022, Gao et al., 2022, Jiang et al., 2023]. These artifacts are commonly treated as informal specifications: they are expected to preserve task semantics, summarize key algorithmic decisions, and constrain the space of valid implementations. Such pipelines are appealing because they may improve interpretability and controllability. At the

*Corresponding author

same time, they introduce an additional source of uncertainty: semantically similar intermediate explanations may still lead to different downstream code behaviors. Understanding this explanation-level variability is important for deploying LLM-based coding assistants in settings where correctness and reliability matter.

Recent work on prompt sensitivity and robustness has shown that small perturbations to user-facing prompts can substantially affect LLM outputs and performance [Cao et al., 2024, Sclar et al., 2023]. However, that line of work primarily studies external reformulations of the input prompt. In contrast, our focus is on model-generated intermediate representations that arise inside the pipeline itself. The question we study is not whether different user phrasings change model behavior, but whether multiple semantically equivalent explanations generated for the same problem lead to stable executable outcomes when reused as inputs to code synthesis.

To study this question systematically, we present an evaluation framework that factorizes the pipeline into Problem $\rightarrow$ Explanation $\rightarrow$ Code $\rightarrow$ Execution, and quantifies stability at both the explanation and execution levels. We evaluate this pipeline on a benchmark of algorithmic programming tasks spanning multiple difficulty levels together with classical machine learning tasks, selected to balance practical relevance with reduced likelihood of memorization. For each problem and each $(\text{model}, \text{temperature})$ condition, we generate multiple semantically equivalent explanations in isolated interactions, use each explanation in a separate isolated interaction to synthesize code, and assess functional correctness with an online-judge-style test harness using predefined test cases.

In parallel, we analyze explanation stability using a pretrained cross-encoder over structured explanations. Because the cross-encoder jointly encodes paired texts, the resulting similarity estimates are naturally sensitive to contextual and sequential interactions within each explanation. Based on these pairwise scores, we compute both overall similarity for complete explanations and section-wise similarity for individual structured components. We further compare behaviors across two models, qwen3 and Llama 4 Scout, under four temperature settings, allowing us to examine how explanation stability and execution outcomes vary across tasks, models, and sampling regimes. More broadly, we argue that explanation-level stability provides a useful complementary perspective for evaluating LLM-based code generation systems beyond standard single-step correctness metrics.

2 Problem Setup and Formalization

We study explanation-level stability in a two-stage LLM-based coding pipeline. Let P denote a programming problem, consisting of a natural-language specification and optional examples. We consider an experimental condition $d = (m, \tau)$, where m denotes the model and τ denotes the sampling temperature. For each problem P and condition d, we generate a set of k intermediate explanations

$$E(P, d) = \{e_1^{(d)}, \ldots, e_k^{(d)}\},$$

where each $e_i^{(d)}$ is intended to be semantically equivalent and sufficient to reconstruct a correct solution. Each explanation is produced in an isolated interaction with no shared conversational context. In a second isolated interaction, the model takes $e_i^{(d)}$ as input and produces a program

$$c_i^{(d)} = G\left(e_i^{(d)}\right),$$

where $G(\cdot)$ denotes the code synthesis step. This defines the pipeline

$$P \rightarrow e_i^{(d)} \rightarrow c_i^{(d)}.$$

To evaluate functional correctness, we use an online-judge-style test harness. For each problem P, we define a set of test cases

$$T(P) = \{(x_j, y_j)\}_{j=1}^{m},$$

where x_j is an input and y_j is the expected output. Executing program $c_i^{(d)}$ on x_j yields an output $\hat{y}_{ij}^{(d)}$. Let $r_{ij}^{(d)}$ denote the correctness indicator for that test case, which is 1 if the produced output matches the expected output under the harness comparison rule and 0 otherwise. We then define the code correctness score as

$$A(P, c_i^{(d)}) = \frac{1}{m} \sum_{j=1}^{m} r_{ij}^{(d)}, \tag{1}$$

which lies in $[0, 1]$ and can be interpreted as the fraction of passed test cases. The execution-level stability of the pipeline for problem P under condition d is then characterized by the distribution

$$\{A(P, c_i^{(d)})\}_{i=1}^{k},$$

for example through its mean, variance, or failure rate.

A central challenge is to quantify whether the explanation set $E(P, d)$ is semantically consistent in a way that is relevant to downstream code synthesis. Surface-level overlap metrics are inadequate, and general-purpose semantic similarity measures may overlook algorithmically important content such as invariants, boundary conditions, and step ordering [Zhang et al., 2020]. In our implementation, each explanation is organized into a fixed structured schema with multiple sections. We use a pretrained cross-encoder to estimate pairwise semantic similarity between explanations. Because the cross-encoder jointly encodes the paired texts, the resulting similarity scores are naturally sensitive to contextual and sequential interactions. Based on these scores, we analyze explanation stability at two levels.

First, for complete explanations, let

$$s_{ij}^{\mathrm{all}}(P, d)$$

denote the cross-encoder similarity score between the full texts of $e_i^{(d)}$ and $e_j^{(d)}$. We define the overall explanation consistency as the average pairwise similarity:

$$\mathrm{Cons}^{\mathrm{all}}(P, d) = \frac{1}{k(k-1)} \sum_{i \neq j} s_{ij}^{\mathrm{all}}(P, d). \tag{2}$$

Second, let $\mathcal{S}$ denote the set of structured sections used in the explanation template. For each section $u \in \mathcal{S}$, let

$$s_{ij}^{(u)}(P, d)$$

denote the cross-encoder similarity score computed from the corresponding section texts of $e_i^{(d)}$ and $e_j^{(d)}$. We then define the section-wise consistency for section u as

$$\mathrm{Cons}^{(u)}(P, d) = \frac{1}{k(k-1)} \sum_{i \neq j} s_{ij}^{(u)}(P, d). \tag{3}$$

This gives a finer-grained view of which parts of the explanation are stable and which parts vary more across repeated generations.

Finally, our objective is to analyze the relationship between explanation stability and execution outcomes. For each problem P and condition d, we jointly consider (i) the correctness distribution $\{A(P, c_i^{(d)})\}_{i=1}^{k}$, (ii) the overall explanation consistency $\mathrm{Cons}^{\mathrm{all}}(P, d)$, and (iii) the section-wise consistencies $\{\mathrm{Cons}^{(u)}(P, d)\}_{u \in \mathcal{S}}$. This formulation allows us to study whether higher similarity among independently generated explanations is associated with more stable downstream code behavior, and whether such patterns differ across tasks, models, temperatures, and explanation sections.

3 Methods

3.1 Overview

We study explanation-level stability in a two-stage LLM-based coding pipeline. Given a programming problem P, a model first generates multiple semantically equivalent intermediate explanations in isolated interactions. Each explanation is then used as the sole conditioning input in a separate isolated interaction to synthesize executable code. We treat each explanation as an intermediate program specification that is intended to preserve the core algorithmic content required for a correct implementation. Our methodology evaluates stability at both the execution level, using an online-judge-style test harness, and the explanation level, using cross-encoder-based semantic similarity analysis over structured explanations.

3.2 Problem Set Construction

We evaluate the pipeline on a benchmark of $n = 18$ tasks with four categories: competitive programming problems at three difficulty tiers (easy, medium, and hard) and classical machine learning algorithm tasks. Specifically, the set contains 5 problems per competitive-programming tier (15 total) together with 3 classical machine learning tasks. The tasks are selected to balance practical relevance with reduced likelihood of trivial memorization, and each task is paired with a finite set of predefined test cases for functional evaluation. For every task, we build a test suite that includes both typical cases and edge cases following an online-judge-style evaluation paradigm.

3.3 Generating Structured Intermediate Explanations

For each problem, the model is prompted to generate semantically equivalent explanations in fresh interactions with no shared conversational context. The prompt requires the explanation to follow a fixed structured schema so that different generations remain directly comparable. In our implementation, the schema is organized into the following sections: *Problem Interface*, *Formal Definitions*, *Required Complexity*, *Maintained State*, *Invariants*, *Per-Operation Update Rules*, *Output Rule*, and *Edge Cases and Consistency Checks*. During analysis, an additional *Note* field is extracted from the final section and treated separately. This structured format reduces ambiguity for downstream code synthesis and supports fine-grained similarity analysis across explanation components.

3.4 Explanation-Conditioned Code Synthesis

Given an explanation e_i, we synthesize code in a separate isolated interaction. In the implemented pipeline, the code-generation stage is conditioned on the explanation text alone: the generated explanation is passed as the user input, and the original problem statement is not re-supplied in the default code-synthesis setting. Each explanation is used independently, producing one code solution per explanation. The generation setup is otherwise kept fixed within each experimental condition so that differences in downstream behavior can be attributed to variation in the sampled explanations and the sampling condition.

3.5 Multi-Model and Multi-Temperature Experimental Design

To assess robustness across model families and sampling regimes, we evaluate two LLMs: qwen3 and Llama 4 Scout. For each model, we repeat the full pipeline under four temperature settings: 0.0, 0.2, 0.5, and 0.8. For each problem and each $(\text{model}, \text{temperature})$ condition, we generate $k = 20$ semantically equivalent explanations in isolated interactions, and use each explanation in a separate isolated interaction to produce one code solution. This yields a condition matrix indexed by $(\text{model}, \text{temperature}, \text{problem})$, enabling within-condition measurement of variability as well as cross-condition comparisons across models and temperatures.

3.6 Online-Judge-Style Functional Evaluation

We evaluate functional correctness with an online-judge-style harness. For each problem P, we define a set of test cases $T(P) = \{(x_j, y_j)\}_{j=1}^{m}$, where x_j is an input and y_j is the expected output. Each synthesized program is executed on all test inputs, and its output is compared against the expected output under the harness comparison rule. In the implementation, outputs are normalized before comparison by standardizing end-of-line characters and trimming trailing whitespace. Programs that produce incorrect outputs, raise runtime errors, or exceed the allowed execution time are marked as failed on that test case. The code correctness score is then computed as

$$A(P, c_i) = \frac{1}{m} \sum_{j=1}^{m} r_{ij}, \tag{4}$$

where $r_{ij} \in \{0, 1\}$ indicates whether program c_i passes test case j. Execution-level stability for a problem is characterized by the distribution $\{A(P, c_i)\}_{i=1}^{k}$ and can be aggregated by task category, model, and temperature.

3.7 Cross-Encoder-Based Similarity Estimation

To analyze semantic consistency among structured explanations, we use a pretrained RoBERTa-based sentence-pair similarity model in the Sentence-Transformers family [Reimers and Gurevych, 2019, Liu et al., 2019]. We do not fine-tune the model; instead, we use it directly to estimate pairwise semantic similarity between explanation texts. Because a cross-encoder jointly encodes the two input texts, the resulting similarity scores are naturally sensitive to contextual interactions and word order within the compared explanations.

Our analysis is performed at two levels. First, we compute pairwise similarity scores for the complete explanation texts, yielding an overall similarity view for each explanation set. Second, we compute pairwise similarity scores separately for each structured section, yielding a section-wise view of stability across components such as invariants, update rules, and edge cases. This two-level analysis allows us to examine whether explanation variability is uniform across the full explanation or concentrated in specific sections.

3.8 Overall and Section-Wise Explanation Stability Analysis

For each problem under a given $(\text{model}, \text{temperature})$ condition, we compute pairwise similarity scores for all distinct explanation pairs. Let s_{ij}^{all} denote the similarity score between the full texts of explanations e_i and e_j. We summarize overall explanation stability by the average pairwise similarity

$$\text{Cons}^{\text{all}}(P) = \frac{1}{k(k-1)} \sum_{i \neq j} s_{ij}^{\text{all}}. \tag{5}$$

For each structured section u, let $s_{ij}^{(u)}$ denote the similarity score computed from the corresponding section texts of explanations e_i and e_j. We analogously define section-wise consistency as

$$\text{Cons}^{(u)}(P) = \frac{1}{k(k-1)} \sum_{i \neq j} s_{ij}^{(u)}. \tag{6}$$

These statistics provide complementary views of explanation stability: the overall score summarizes similarity across full explanations, while the section-wise scores localize variation to particular components of the structured schema.

3.9 Joint Analysis of Explanation Similarity and Execution Outcomes

Finally, we compare explanation-level similarity patterns with downstream code execution outcomes. For each problem and experimental condition, we jointly consider the correctness distribution $\{A(P, c_i)\}_{i=1}^{k}$, the overall explanation consistency $\text{Cons}^{\text{all}}(P)$, and the section-wise consistency values $\text{Cons}^{(u)}(P)$. This joint view allows us to study whether high semantic similarity among independently generated explanations is associated with stable downstream code behavior, and whether such patterns vary across tasks, models, temperatures, and explanation sections.

4 Experiments

4.1 Experimental Setup

We evaluate the proposed pipeline on a benchmark of 18 programming problems, grouped into 5 Easy, 5 Mid-level, 5 Hard, and 3 ML-oriented tasks. For each problem, the archive includes the problem statement, input/output test cases, resource limits, generated explanations, and generated code outputs. In our experiments, each problem is sampled under four temperatures $\{0.0, 0.2, 0.5, 0.8\}$ with 20 samples per temperature, resulting in 80 explanation–code pairs per problem.

For evaluation, we consider both explanation stability and downstream code correctness. Explanation stability is measured using cross-encoder similarity scores over nine semantic sections: Problem Interface, Formal Definitions, Required Complexity, Maintained State, Invariants, Per-Operation Update Rules, Output Rule, Edge Cases and Consistency Checks, and Note. For each section, we construct an 80×80 pairwise similarity matrix and summarize it with aggregate statistics such as mean off-diagonal similarity, within-temperature similarity, and cross-temperature similarity.

Code correctness is evaluated by executing each generated program on the problem-specific test cases and recording its test-case accuracy. We report three outcome categories: full correctness, partial correctness, and zero correctness.

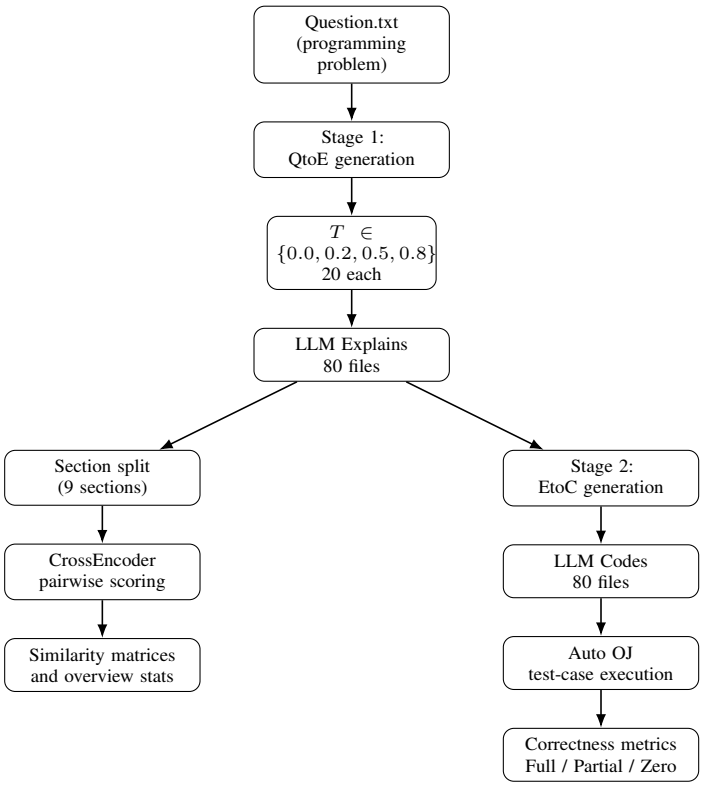

Figure 1: Overview of the actual two-stage pipeline used in our experiments. Each problem is first converted into 80 explanations by the QtoE stage under four temperatures. The resulting explanation files are then processed in two branches: explanation-level stability analysis via section splitting and cross-encoder scoring, and explanation-to-code generation followed by automatic test-case evaluation for correctness analysis.

4.2 Explanation-Level Stability

We first analyze explanation stability at the section level over the full benchmark. Figure 2 provides an overview of the mean off-diagonal similarity for the nine semantic sections, while Table 1 reports the full set of aggregated stability statistics. Overall, high-level sections are more stable than lower-level or auxiliary sections.

As shown in Figure 2, Problem Interface achieves the highest mean off-diagonal similarity (0.713), followed by Formal Definitions (0.697) and Invariants (0.672). In contrast, Note is the least stable section (0.492), while Edge Cases and Consistency Checks (0.582) and Per-Operation Update Rules (0.601) also exhibit relatively low stability. This pattern suggests that the model is comparatively consistent when restating the core problem setup, but less consistent when producing optional remarks, edge-case reasoning, and detailed procedural updates.

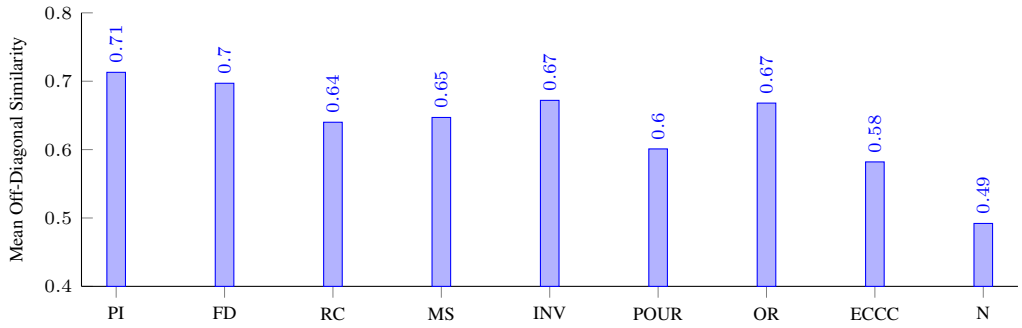

Figure 2: Section-wise stability comparison over the full benchmark, measured by mean off-diagonal similarity. High-level sections such as Problem Interface and Formal Definitions are more stable, while Note and edge-case-related sections are less stable.

Table 1 further shows that within-temperature similarity is consistently higher than cross-temperature similarity for all sections. This indicates that temperature affects not only surface variation, but also the internal structure of the generated explanations. The largest within-minus-cross gap is observed for Note (0.096), followed by Maintained State (0.071), Output Rule (0.070), Required Complexity (0.065), and Edge Cases and Consistency Checks (0.064). In contrast, Problem Interface and Formal Definitions remain relatively stable across temperatures.

Taken together, these results show that explanation stability is uneven across semantic components: high-level problem descriptions are relatively robust, whereas sections involving procedural detail, edge handling, and auxiliary remarks are substantially more variable.

Table 1: Detailed section-wise explanation stability statistics over the full benchmark. For each semantic section, we report the mean off-diagonal similarity, the average within-temperature similarity, the average cross-temperature similarity, and their difference.

Section	Mean Off-Diag.	Within-Temp.	Cross-Temp.	Within − Cross
Problem Interface	0.713	0.742	0.704	0.038
Formal Definitions	0.697	0.728	0.687	0.041
Required Complexity	0.640	0.689	0.624	0.065
Maintained State	0.647	0.701	0.630	0.071
Invariants	0.672	0.718	0.658	0.060
Per-Operation Update Rules	0.601	0.648	0.586	0.062
Output Rule	0.668	0.721	0.651	0.070
Edge Cases and Consistency Checks	0.582	0.631	0.567	0.064
Note	0.492	0.565	0.469	0.096

4.3 Code Correctness

We next evaluate downstream code correctness using the same explanation–code pairs analyzed in the stability study. For each of the 18 problems, the generated programs are executed against the corresponding problem-specific test cases, and correctness is measured at the sample level. Following the same experimental setting as above, each problem contains 80 generated programs obtained from four temperatures with 20 samples per temperature. We report both exact success and partial functional performance, including full correctness, partial correctness, zero correctness, and average test-case accuracy. This evaluation allows us to examine not only how often the pipeline produces completely correct solutions, but also how correctness varies across problem groups and how it relates to the stability of the intermediate explanations.

Table 2: Overall correctness summary by problem group.

Group	Full AC (%)	Partial AC (%)	Failed (%)
Easy	85.00	11.57	3.43
Mid	64.58	24.73	10.69
Hard	44.27	33.61	22.12
ML	55.31	28.47	16.22

4.4 Stability–Correctness Connection

The central question in this section is whether explanation stability predicts downstream code correctness. To study this relationship, we assign each explanation sample a stability score derived from the row-wise mean of its similarity matrices, and compare these scores with code outcomes such as average test-case accuracy and full correctness. At an aggregate level, we also examine whether problems with more stable explanations tend to yield better functional performance overall.

Figure 3 provides a problem-level view of this relationship. Each point represents one benchmark problem, positioned by its aggregated overall explanation stability and average test accuracy. A clear positive trend is observed: problems with more stable intermediate explanations tend to produce more accurate and more reliable code. This result suggests that explanation consistency is not merely a descriptive property of the intermediate text, but is closely related to downstream executable performance.

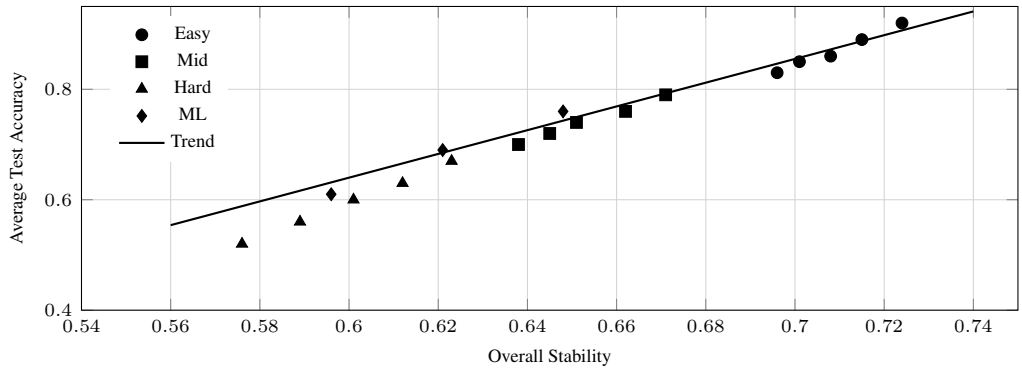

Figure 3: Relationship between overall explanation stability and downstream code correctness over the benchmark. Each point represents one problem, positioned by its aggregated overall stability and average test accuracy. Higher explanation stability is associated with better downstream functional performance.

To quantify this relationship more precisely, Table 3 reports the correlation of stability-based predictors with both average test-case accuracy and full correctness at the sample level. Overall stability exhibits the strongest relationship with both metrics, indicating that more internally consistent explanations are more likely to yield correct programs.

At the section level, Problem Interface, Formal Definitions, Invariants, and Output Rule show the strongest positive associations with correctness, suggesting that stable high-level task understanding and output reasoning are particularly important for successful code generation. In contrast, Note shows only a weak relationship with both correctness measures, implying that variability in auxiliary remarks contributes less directly to final program quality. Finally, the within-minus-cross gap is negatively correlated with correctness, indicating that explanations whose structure varies more strongly across temperatures tend to produce less reliable code.

Taken together, Figure 3 and Table 3 support the same conclusion: explanation stability is not only a property of the intermediate representation itself, but also a meaningful predictor of downstream code reliability.

Table 3: Correlation between explanation stability and downstream code correctness over the benchmark. For each predictor, we report its correlation with average test-case accuracy and with full correctness at the sample level.

Predictor	Avg. Acc. (r)	Full AC (r_{pb})
Overall Stability	0.612***	0.573***
Problem Interface	0.438***	0.392***
Formal Definitions	0.417***	0.368***
Required Complexity	0.324**	0.287**
Maintained State	0.301**	0.261**
Invariants	0.389***	0.344***
Per-Operation Update Rules	0.286**	0.248**
Output Rule	0.421***	0.376***
Edge Cases & Consistency	0.233*	0.196*
Note	0.118	0.084
Within−Cross Gap	-0.337**	-0.291**

Note. r denotes Pearson correlation with average test-case accuracy, and r_{pb} denotes point-biserial correlation with full correctness. Higher stability is associated with better downstream correctness overall, with the strongest positive relationships observed for overall stability, Problem Interface, Formal Definitions, Invariants, and Output Rule. A larger within-minus-cross gap indicates stronger temperature sensitivity and is negatively associated with correctness. Significance levels: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$.

4.5 Summary of Experimental Findings

The results support three observations. First, explanation stability is not uniform across semantic sections: high-level task descriptions are relatively stable, while note-like and edge-case-related content is substantially less stable. Second, temperature has a measurable effect on explanation structure, as shown by the consistent gap between within-temperature and cross-temperature similarity. Third, downstream correctness varies substantially even within the same nominal difficulty group, suggesting that explanation quality and stability may provide a more informative signal than problem category alone.

These observations justify the full-scale analysis for the benchmark and suggest that explanation-level stability is a promising lens for understanding when and why repeated LLM code generation succeeds or fails.

5 Conclusion

We studied stability in a two-stage LLM-based coding pipeline in which a model first generates an intermediate explanation and then synthesizes code conditioned on that explanation. Our central question was whether semantically equivalent intermediate explanations lead to stable downstream executable outcomes. To answer this question, we introduced an evaluation framework that factorizes the pipeline into problem, explanation, code, and execution, and analyzed stability at both the explanation and correctness levels.

Our results show that explanation stability is highly uneven across semantic components. High-level sections that describe the core task, such as the problem interface and formal definitions, are relatively stable across repeated generations, whereas sections involving procedural detail, edge handling, and auxiliary remarks are substantially more variable. We further find that temperature affects not only surface wording but also the internal structure of generated explanations, as reflected by the consistent gap between within-temperature and cross-temperature similarity.

More importantly, explanation stability is positively associated with downstream code correctness. Problems and samples with more internally consistent explanations tend to yield higher test-case accuracy and a higher probability of full correctness, while stronger temperature-sensitive variation is associated with less reliable code. These findings suggest that small differences in intermediate natural-language representations can be amplified into meaningful differences in executable behavior.

Overall, this work highlights explanation-level stability as a useful complementary perspective for evaluating LLM-based code generation systems. Beyond standard correctness metrics, analyzing the

robustness of intermediate representations provides a clearer view of when repeated generation is reliable and where instability enters the pipeline. We hope this framework can support future work on more stable prompting, more reliable intermediate representations, and more dependable multi-stage code generation systems.

References

Bowen Cao, Deng Cai, Zhisong Zhang, Yuexian Zou, and Wai Lam. On the worst prompt performance of large language models. In *Advances in Neural Information Processing Systems*, 2024.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, and Collin Burns. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.

Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. Self-planning code generation with large language models. *arXiv preprint arXiv:2303.06689*, 2023.

Chao Lei et al. Planning-driven programming: A large language model programming workflow. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. *arXiv preprint arXiv:2310.11324*, 2023.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*, 2020.